ByteMart Digital

THE COMPLETE GUIDE

JSON & XML

A Masterclass for Java Developers

A complete, example-driven masterclass on JSON and XML for Java developers — from syntax and Jackson to schemas, JAXB, XPath, XSLT, interop, and security.

24	120+	13	6
Chapters	Code Examples	Diagrams	Modules

© 2026 ByteMart Digital · All Rights Reserved

ByteMart Digital

JSON & XML Masterclass

First Edition · 2026

Copyright © 2026 ByteMart Digital. All rights reserved.
No part of this publication may be reproduced, distributed, or transmitted in any form or by any means — including photocopying, recording, screenshotting, or other electronic or mechanical methods — without the prior written permission of the publisher, except for brief quotations used in reviews and certain other non-commercial uses permitted by copyright law.

This product is licensed for personal use by the purchaser only. Sharing, reselling, or redistributing this file in whole or in part is strictly prohibited and constitutes a violation of copyright law.

Disclaimer
The information in this book is provided in good faith and for educational purposes. While every effort has been made to ensure accuracy, the author and publisher assume no responsibility for errors, omissions, or outcomes arising from the use of the code or information contained herein. All code examples are provided "as is".

Trademarks
Jackson, JAXB, and other product names, logos, and brands referenced are the property of their respective owners and are used for identification purposes only. This guide is an independent educational work.

Published by ByteMart Digital

Web: bytemartdigital.com
Email: contact@bytemartdigital.in
Instagram: instagram.com/bytemartdigital.in
Telegram: t.me/bytemartcodingcommunity

Contents

1. Why Data Formats Matter 4
2. JSON Syntax & Structure 7
3. JSON Data Types in Depth 10
4. JSON vs XML: Choosing a Format 12
5. Jackson: The Java JSON Library 15
6. Serialization: Objects to JSON 18
7. Deserialization: JSON to Objects 20
8. Jackson Annotations 22
9. Trees, Streaming & Custom Serializers 25
10. JSON Schema: Validating Structure 28
11. Querying with JSONPath 31
12. JSON in Spring Boot REST APIs 34
13. Dates, Enums & Polymorphism 36
14. XML Syntax & Structure 39
15. Well-Formed vs Valid XML 42
16. XML Schema (XSD) 44
17. Namespaces in Depth 47
18. Parsing XML: DOM, SAX & StAX 49
19. JAXB: XML Binding 52
20. Querying with XPath 55
21. Transforming with XSLT 58
22. JSON <-> XML Interop 61
23. Security: XXE & Injection 63
24. Performance & Best Practices 65

MODULE 1 · JSON FUNDAMENTALS

CHAPTER 01

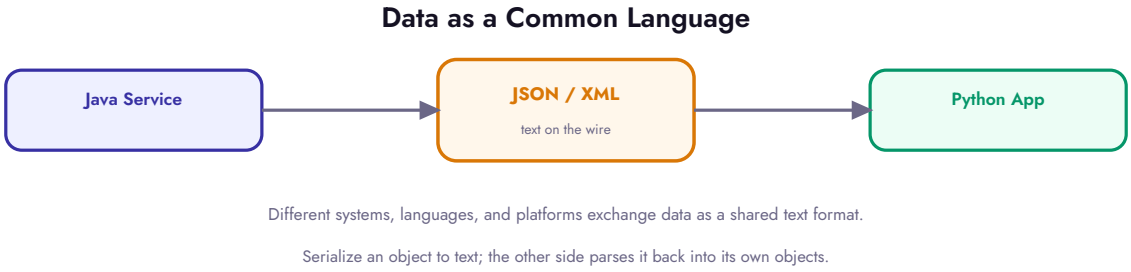
Why Data Formats Matter

Serialization, interchange, and why JSON and XML are the lingua franca of software.

Software constantly moves data between systems — a browser and a server, two microservices, an app and a database. But an in-memory Java object cannot travel over a network. It must be turned into text, sent, and rebuilt. That is **serialization**, and JSON and XML are the two dominant formats for it.

> **Serialization**
> Converting an in-memory object into a portable representation (usually text) that can be stored or transmitted, and later parsed back into an object — possibly in a different language or system.

A common language



Different systems exchange data as a shared text format, each parsing it into its own objects.

The power of JSON and XML is that they are **language-neutral**. A Java service can serialize an object to JSON, send it to a Python or JavaScript client, and that client parses it into its own native structures — neither needs to know the other's language.

Where you meet them

- **REST APIs** — request and response bodies (mostly JSON).
- **Configuration** — files and settings (JSON, XML, YAML).
- **Messaging** — event payloads on brokers.
- **Documents** — structured content and enterprise data exchange (often XML).

What this masterclass covers

You will learn both formats deeply — their syntax, how to read and write them in Java (Jackson, JAXB), how to validate them (JSON Schema, XSD), how to query them (JSONPath, XPath), and how to use them safely and efficiently in real applications.

© 2026 ByteMart Digital

Text is the interchange

Serialization turns objects into portable text so different systems can exchange data. JSON and XML are the two language-neutral formats that dominate APIs, config, messaging, and documents.

© 2026 ByteMart Digital

MODULE 1 · JSON FUNDAMENTALS

CHAPTER 02

JSON Syntax & Structure

Objects, arrays, and the small, strict grammar that makes JSON so easy to parse.

JSON — JavaScript Object Notation — wins on simplicity. Its entire grammar fits on a page, which is exactly why it is so easy for machines and humans alike.

The building blocks

Anatomy of a JSON Object



A JSON object is key-value pairs; values can be strings, numbers, booleans, null, arrays, or nested objects.

JSON is built from two structures: **objects** (unordered key-value pairs in braces) and **arrays** (ordered lists in brackets). Values are strings, numbers, booleans, null, or nested objects and arrays.

```json
{
  "name": "Ada",
  "age": 36,
  "active": true,
  "roles": ["admin", "dev"],
  "address": { "city": "Delhi", "pin": "110001" },
  "manager": null
}
```

The strict rules

- Keys must be **double-quoted strings** — always.
- Strings use double quotes, never single.
- **No trailing commas** and **no comments** in standard JSON.
- Whitespace between tokens is ignored.

WATCH OUT

Standard JSON forbids comments and trailing commas — a common source of parse errors when hand-editing. If you copy JavaScript object literals into JSON, these differences bite.

© 2026 ByteMart Digital

Objects, arrays, six value types

JSON is just objects and arrays holding strings, numbers, booleans, null, and nested structures. Keys are always quoted strings, and there are no comments or trailing commas — strictness makes it easy to parse.

© 2026 ByteMart Digital

MODULE 1 · JSON FUNDAMENTALS

CHAPTER 03

JSON Data Types in Depth

The subtleties of numbers, strings, and null that trip people up.

JSON has only a handful of types, but each has subtleties worth knowing — especially numbers, which behave differently than many expect.

The value types

Type	Example	Notes
string	"hello"	double quotes, unicode escapes
number	42, 3.14, 1e6	no distinction int vs float
boolean	true / false	lowercase only
null	null	explicit absence
object	{ }	key-value pairs
array	[ ]	ordered list

Numbers have no type distinction

JSON has one number type — there is no separate integer or float. It also has no native support for very large integers or high-precision decimals, so a 64-bit id or a monetary value can lose precision if a parser maps it to a floating-point double.

WATCH OUT

Large integers (like a 64-bit id) and precise decimals (like money) can lose precision when parsed as doubles. Represent them as **strings** in JSON, or use a parser configured for `BigDecimal/BigInteger`.

Strings and escaping

Strings are Unicode, with backslash escapes for quotes, backslashes, and control characters (`\n`, `\t`, `\uXXXX`). A literal double quote or backslash inside a string must be escaped.

null vs missing

A key set to `null` is present with an explicit null value; a key that is simply absent is different. Many bugs come from conflating the two — decide deliberately which your API means.

Few types, sharp edges

JSON's value types are simple, but numbers have no int/float split and can lose precision — use strings for ids and money. Distinguish an explicit null from a missing key deliberately.

© 2026 ByteMart Digital

MODULE 1 · JSON FUNDAMENTALS

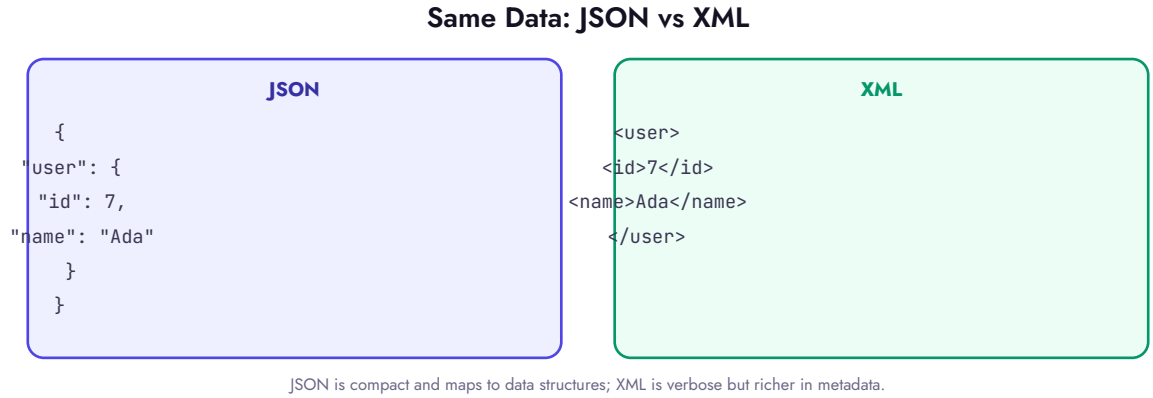
CHAPTER 04

JSON vs XML: Choosing a Format

An honest comparison so you pick the right format for each job.

JSON and XML solve the same core problem — structured data interchange — but with different trade-offs. Neither is universally better; each fits certain jobs.

The same data, two forms



The same data is compact in JSON and more verbose but metadata-rich in XML.

How they compare

	JSON	XML
Verbosity	compact	verbose (tags repeat)
Data mapping	maps to structures directly	document-oriented
Attributes	no (only values)	yes (element attributes)
Comments	no	yes
Schema	JSON Schema	XSD / DTD (mature)
Namespaces	no	yes

When to use JSON

- REST APIs and web/mobile clients — the modern default.
- Compact payloads where size and speed matter.
- Data that maps naturally to objects and arrays.

When XML still shines

- Document-centric content with mixed text and markup.

© 2026 ByteMart Digital

- Enterprise and legacy systems (SOAP, many standards).
- When you need namespaces, rich schemas, or transformation (XSLT).

Right format for the job

JSON is compact and object-like — the default for APIs. XML is verbose but richer in metadata, namespaces, and mature tooling — preferred for documents and enterprise standards. Choose per use case.

© 2026 ByteMart Digital

MODULE 2 · JSON IN JAVA

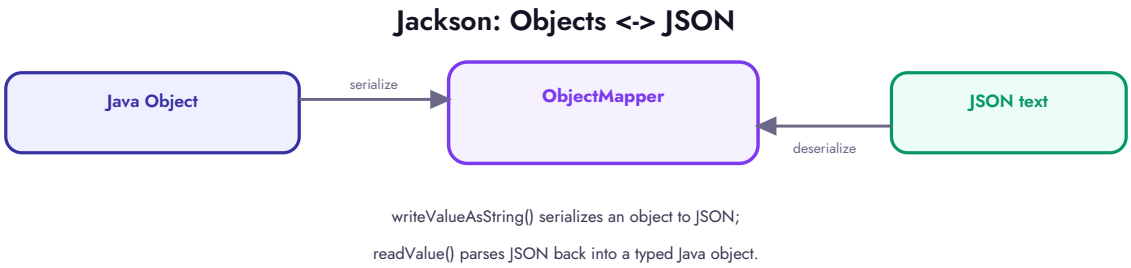
CHAPTER 05

Jackson: The Java JSON Library

The de facto standard for JSON in Java, and the ObjectMapper at its heart.

In the Java world, **Jackson** is the standard library for JSON — it powers Spring Boot's JSON handling out of the box. Its central class, `ObjectMapper`, converts between Java objects and JSON.

Objects to JSON and back



ObjectMapper serializes a Java object to JSON and deserializes JSON back into a typed object.

```java
ObjectMapper mapper = new ObjectMapper();

// serialize: object -> JSON
String json = mapper.writeValueAsString(user);

// deserialize: JSON -> object
User user = mapper.readValue(json, User.class);
```

Why Jackson dominates

- **Fast** — among the fastest JSON libraries for Java.
- **Flexible** — annotations, modules, and three processing APIs.
- **Ubiquitous** — the default in Spring Boot, so you already use it.
- **Extensible** — modules for `java.time`, Kotlin, and more.

Configuring the mapper

The `ObjectMapper` is highly configurable and **thread-safe once configured** — create and configure one, then reuse it. In Spring Boot, a pre-configured mapper is available as a bean.

> **TIP**
> Register modules for special types — for example the `JavaTimeModule` so Jackson handles `LocalDate` and `Instant` correctly instead of as timestamps.

ObjectMapper is the workhorse

Jackson is the standard Java JSON library, and its thread-safe ObjectMapper serializes objects to JSON and deserializes back. Configure one and reuse it; Spring Boot provides one for you.

© 2026 ByteMart Digital

MODULE 2 · JSON IN JAVA

CHAPTER 06

Serialization: Objects to JSON

Turning Java objects into JSON, and controlling what and how fields are written.

Serialization is writing a Java object as JSON. Jackson does this automatically for POJOs, and gives you fine control over what is included and how it looks.

The default behaviour

By default Jackson serializes every public getter (or public field) as a JSON property, using the property name. A `User` with `getName()` and `getEmail()` becomes `{"name":...,"email":...}`.

```java
record User(String name, String email, int age) {}

String json = mapper.writeValueAsString(
    new User("Ada", "ada@x.com", 36));
// {"name":"Ada","email":"ada@x.com","age":36}
```

Pretty printing

```java
mapper.writerWithDefaultPrettyPrinter()
      .writeValueAsString(user);
```

Controlling inclusion

Use `@JsonInclude` to omit empty or null fields, keeping payloads lean. This is common for API responses where absent means 'not set'.

```java
@JsonInclude(JsonInclude.Include.NON_NULL)
record User(String name, String email, String phone) {}
// phone omitted entirely when null
```

Writing to different targets

Beyond a `String`, Jackson can write directly to a file, an `OutputStream`, or a `byte[]` — useful for large payloads you do not want to hold in memory as a string.

Objects out, controlled
Jackson serializes POJOs to JSON via getters by default. Control the output with pretty printing, @JsonInclude to drop nulls, and write straight to streams or files for large payloads.

© 2026 ByteMart Digital

MODULE 2 · JSON IN JAVA

CHAPTER 07

Deserialization: JSON to Objects

Parsing JSON into typed Java objects, including collections and generics.

Deserialization is the reverse — reading JSON into Java objects. Jackson binds JSON properties to fields by name, constructing your objects for you.

Binding to a class

```java
String json = "{\"name\":\"Ada\",\"age\":36}";
User user = mapper.readValue(json, User.class);
```

Jackson matches JSON keys to constructor parameters or setters/fields. For records and immutable classes, it binds through the constructor.

Collections and generics

Because of type erasure, deserializing into a generic collection needs a `TypeReference` so Jackson knows the element type.

```java
List<User> users = mapper.readValue(json,
    new TypeReference<List<User>>() {});
```

Unknown properties

By default, JSON containing a property with no matching field causes an error. This is strict but can be inconvenient when an API adds fields — configure Jackson to ignore unknowns when appropriate.

```java
mapper.configure(
    DeserializationFeature.FAIL_ON_UNKNOWN_PROPERTIES, false);
// or @JsonIgnoreProperties(ignoreUnknown = true) on the class
```

WATCH OUT

Ignoring unknown properties makes clients resilient to new fields, but can also hide typos and contract drift. Decide deliberately whether strictness or tolerance suits each consumer.

JSON in, typed out

Jackson binds JSON to classes by matching keys to constructor params or fields; use TypeReference for generic collections. Choose whether to fail or ignore on unknown properties based on the consumer.

© 2026 ByteMart Digital

MODULE 2 · JSON IN JAVA

CHAPTER 08

Jackson Annotations

The annotations that customise how fields map between Java and JSON.

Real APIs rarely match your Java field names exactly. Jackson annotations bridge the gap — renaming, ignoring, and reshaping properties without changing your logic.

The essential annotations

Annotation	Effect
@JsonProperty("x")	map field to JSON key 'x'
@JsonIgnore	exclude a field entirely
@JsonInclude	control null/empty inclusion
@JsonFormat	format dates and numbers
@JsonAlias	accept alternative input names
@JsonCreator	mark a constructor for binding

java

```
public class User {
    @JsonProperty("user_name")
    private String name;          // JSON key is user_name

    @JsonIgnore
    private String passwordHash;  // never serialized

    @JsonFormat(pattern = "yyyy-MM-dd")
    private LocalDate joined;
}
```

Renaming strategies

Rather than annotating every field, apply a **naming strategy** (like snake_case) globally so all Java `camelCase` fields map to `snake_case` JSON keys automatically.

java

```
mapper.setPropertyNamingStrategy(
    PropertyNamingStrategies.SNAKE_CASE);
```

WATCH OUT

Always `@JsonIgnore` sensitive fields like passwords and tokens — accidental serialization of a secret into an API response is a real and common leak.

© 2026 ByteMart Digital

Map fields cleanly

Jackson annotations rename (@JsonProperty), hide (@JsonIgnore), format (@JsonFormat), and reshape the JSON mapping. Use a naming strategy for consistent casing, and always ignore secrets.

© 2026 ByteMart Digital

MODULE 2 · JSON IN JAVA

CHAPTER 09

Trees, Streaming & Custom Serializers

Jackson's three processing models and when to reach beyond simple data binding.

Data binding to POJOs handles most needs, but Jackson offers three processing models. Knowing all three lets you handle dynamic data and huge files efficiently.

The three APIs



Bind to POJOs by default; drop to the tree model for dynamic data, streaming for huge files.

Data binding for POJOs, the tree model for dynamic data, and streaming for the largest files.

The tree model

When the JSON shape is dynamic or unknown, read it into a `JsonNode` tree and navigate it like a document — no fixed class required.

```java
JsonNode root = mapper.readTree(json);
String city = root.path("address").path("city").asText();
```

Streaming for huge files

For very large documents, the streaming API (`JsonParser`/`JsonGenerator`) processes one token at a time with minimal memory — you never hold the whole structure at once.

Custom serializers

When a type needs special handling, write a custom serializer or deserializer and register it — for example to output money in a specific format, or to parse a legacy date shape.

© 2026 ByteMart Digital

```java
public class MoneySerializer extends JsonSerializer<Money> {
    public void serialize(Money m, JsonGenerator gen,
            SerializerProvider p) throws IOException {
        gen.writeString(m.amount() + " " + m.currency());
    }
}
```

Bind, tree, or stream

Use data binding for POJOs, the JsonNode tree model for dynamic data, and streaming for huge files with minimal memory. Register custom (de)serializers for types that need special handling.

MODULE 3 · ADVANCED JSON

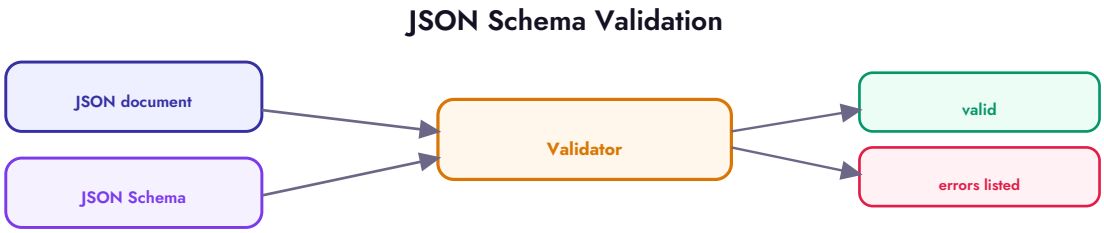
CHAPTER 10

JSON Schema: Validating Structure

Describing and enforcing the shape of JSON documents.

JSON's flexibility is a double-edged sword — nothing stops a client sending the wrong shape. **JSON Schema** is a standard for describing the expected structure and validating documents against it.

How validation works



A schema describes the required shape; the validator checks a document against it.

A schema describes the required shape; a validator checks a document and reports errors.

A simple schema

```json
{
  "type": "object",
  "required": ["name", "age"],
  "properties": {
    "name": { "type": "string", "minLength": 1 },
    "age":  { "type": "integer", "minimum": 0 },
    "email": { "type": "string", "format": "email" }
  }
}
```

Common keywords

- `type` — the expected data type.
- `required` — which properties must be present.
- `minimum`/`maximum`, `minLength`/`maxLength` — bounds.
- `pattern` — a regex the string must match.
- `enum` — an allowed set of values.

Validating in Java

Libraries such as the networknt `json-schema-validator` load a schema and validate documents, returning a list of violations you can surface to the client.

Enforce the shape

JSON Schema describes the required structure — types, required fields, bounds, and patterns — and a validator checks documents against it, catching malformed data at the boundary before it reaches your logic.

© 2026 ByteMart Digital

MODULE 3 · ADVANCED JSON

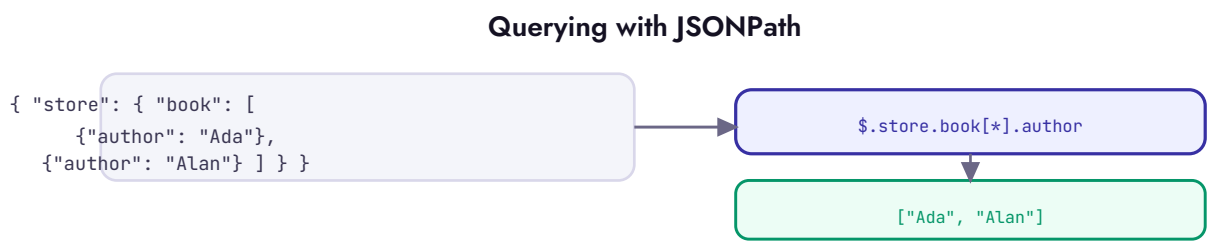
CHAPTER 11

Querying with JSONPath

Extracting values from JSON with path expressions, without full deserialization.

Sometimes you need one value from a large JSON document without mapping the whole thing to objects. **JSONPath** is a query language — like XPath for XML — that plucks values by a path expression.

A path expression



A path expression selects values from a JSON document, like XPath for XML.

A JSONPath expression selects matching values from a JSON document.

```text
$.store.book[*].author      // all book authors
$.store.book[0].title       // first book's title
$..price                    // every price, at any depth
$.book[?(@.price < 10)]     // books cheaper than 10
```

The syntax

Symbol	Meaning
$	the root document
.key or ['key']	a child property
[*]	all array elements
..	recursive descent (any depth)
[?(...)]	a filter expression

In Java

The Jayway JsonPath library evaluates expressions against a document, returning the matched values — handy for tests, extraction, and lightweight processing.

```java
String author = JsonPath.read(json,
    "$.store.book[0].author");
```

TIP

JSONPath is especially useful in API tests — assert on a single value deep in a response without deserializing the entire body into objects.

Pluck values by path

JSONPath selects values from JSON with path expressions — children, wildcards, recursive descent, and filters. It extracts data without full deserialization, and shines in API testing.

MODULE 3 · ADVANCED JSON

CHAPTER 12

JSON in Spring Boot REST APIs

How Spring uses Jackson automatically, and how to shape your API's JSON.

In a Spring Boot REST API you rarely touch `ObjectMapper` directly — Spring wires Jackson in for you. Understanding that integration lets you shape your API's JSON cleanly.

Automatic conversion

Return an object from a `@RestController` method and Spring serializes it to JSON with Jackson; accept a `@RequestBody` and Spring deserializes the incoming JSON into your class — no manual conversion.

```java
@PostMapping("/users")
public User create(@RequestBody CreateUser body) {
    return service.create(body);   // body parsed, result serialized
}
```

Configuring the mapper globally

Customise the JSON behaviour for the whole application through properties or a `Jackson2ObjectMapperBuilderCustomizer` bean — for naming strategy, date format, and inclusion rules.

```properties
spring.jackson.property-naming-strategy=SNAKE_CASE
spring.jackson.default-property-inclusion=non_null
spring.jackson.serialization.write-dates-as-timestamps=false
```

DTOs, not entities

WATCH OUT

Do not serialize JPA entities directly to JSON. Lazy associations trigger surprise queries or errors, and you risk leaking internal fields. Map to a dedicated **DTO** that defines exactly your API contract.

Validation at the boundary

Combine deserialization with Bean Validation (`@Valid` plus constraints like `@NotBlank`) so malformed request bodies are rejected with clear errors before reaching your logic.

Spring does the JSON for you

Spring Boot auto-applies Jackson to serialize responses and deserialize request bodies. Configure it globally via properties, validate at the boundary, and expose DTOs — never JPA entities — as your contract.

MODULE 3 · ADVANCED JSON

CHAPTER 13

Dates, Enums & Polymorphism

The three cases that most often trip up JSON binding in Java — and how to handle them.

Most Java-to-JSON mapping is effortless, but three cases consistently cause trouble: dates and times, enums, and polymorphic types. Handle these well and you have mastered real-world Jackson.

Dates and times

By default Jackson serializes `java.time` types (like `Instant`) as numeric timestamps, which is rarely what you want. Register the `JavaTimeModule` and disable timestamps to get ISO-8601 strings.

```java
mapper.registerModule(new JavaTimeModule());
mapper.disable(SerializationFeature.WRITE_DATES_AS_TIMESTAMPS);
// LocalDate -> "2026-01-15", Instant -> "2026-01-15T10:30:00Z"
```

Enums

Enums serialize to their name by default. Use `@JsonValue` on a method to control the serialized form, and Jackson can deserialize case-insensitively if configured — useful for stable API contracts independent of Java naming.

Polymorphism

When a field can hold one of several subtypes, Jackson needs a hint to pick the right class on the way back. `@JsonTypeInfo` writes a type discriminator into the JSON, and `@JsonSubTypes` lists the possibilities.

```java
@JsonTypeInfo(use = Id.NAME, property = "type")
@JsonSubTypes({
    @JsonSubTypes.Type(value = Card.class, name = "card"),
    @JsonSubTypes.Type(value = Upi.class,  name = "upi")
})
sealed interface Payment permits Card, Upi {}
```

WATCH OUT

Do not enable Jackson's **default typing** to solve polymorphism — it has been the source of serious deserialization vulnerabilities. Use explicit `@JsonSubTypes` instead.

Other libraries

Jackson is the default, but **Gson** (Google) and **JSON-B** (the Jakarta standard) are alternatives with similar concepts. In Spring, stick with Jackson unless you have a specific reason.

© 2026 ByteMart Digital

Master dates, enums, subtypes

Register JavaTimeModule for ISO dates, control enums with @JsonValue, and handle polymorphism with explicit @JsonTypeInfo/@JsonSubTypes — never default typing. These three cases cover most real-world binding pain.

© 2026 ByteMart Digital

MODULE 4 · XML FUNDAMENTALS

CHAPTER 14

XML Syntax & Structure

Elements, attributes, and the rules that make a document valid XML.

XML — eXtensible Markup Language — is older and more verbose than JSON, but richer. Everything is expressed with nested **elements** and their **attributes**.

The anatomy



XML has a prolog, elements with attributes, nested child elements, and text content.

```xml
<?xml version="1.0" encoding="UTF-8"?>
<book id="b1" lang="en">
  <title>XML Basics</title>
  <author>Ada Lovelace</author>
  <price currency="INR">299</price>
</book>
```

The pieces

- **Prolog** — the optional `<?xml ...?>` declaration.
- **Element** — a tag pair with content: `<title>...</title>`.
- **Attribute** — a name-value pair on an element: `id="b1"`.
- **Text** — the content between tags.
- **Root element** — exactly one, wrapping everything.

Elements vs attributes

The same data can often be an element or an attribute. A common guideline: use elements for **data** (the content) and attributes for **metadata** (identifiers, types). Attributes cannot have structure; elements can nest.

Special content

Reserved characters (<, &) must be escaped as entities (`<`, `&`), or wrapped in a `CDATA` section for large blocks of raw text like embedded code.

Elements, attributes, one root

XML nests elements (with optional attributes) under a single root, preceded by an optional prolog. Use elements for data and attributes for metadata, and escape reserved characters or use CDATA.

© 2026 ByteMart Digital

MODULE 4 · XML FUNDAMENTALS

CHAPTER 15

Well-Formed vs Valid XML

Two distinct guarantees — correct syntax versus conformance to a schema.

XML has two separate notions of correctness, and confusing them causes real bugs. A document can be perfectly well-formed yet completely invalid.

Well-formed

A **well-formed** document obeys XML's syntax rules: one root element, every tag closed, proper nesting, quoted attribute values, and escaped special characters. Any XML parser rejects a document that is not well-formed.

- Exactly one root element.
- Every start tag has a matching end tag.
- Elements nest correctly (no overlapping).
- Attribute values are quoted.

Valid

A **valid** document is well-formed and conforms to a declared schema — a DTD or XSD — which specifies the allowed elements, their order, types, and cardinality. Validity is a stronger, optional guarantee.

	Well-formed	Valid
Checks	syntax rules	syntax + schema
Needs a schema	no	yes
Enforced by	any parser	a validating parser

Why the distinction matters

Well-formedness guarantees the document can be parsed; validity guarantees it means what your application expects. A syntactically perfect document with the wrong elements is well-formed but invalid — only schema validation catches it.

Parseable vs correct
Well-formed means the XML obeys syntax rules and can be parsed; valid means it also conforms to a schema (DTD/XSD). Well-formedness is mandatory; validity is a stronger, optional check of structure and content.

© 2026 ByteMart Digital

MODULE 4 · XML FUNDAMENTALS

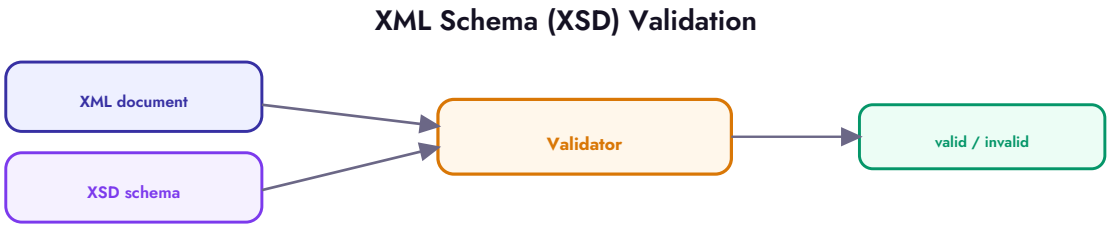
CHAPTER 16

XML Schema (XSD)

Defining and enforcing the structure and types of XML documents.

XSD (XML Schema Definition) is the modern, powerful way to describe what a valid XML document looks like — far richer than the older DTD, and itself written in XML.

Validation against a schema



An XSD defines allowed elements, types, and structure; the parser validates against it.

An XSD defines allowed elements and types; a validating parser checks a document against it.

A small schema

```xml
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="book">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="title" type="xs:string"/>
        <xs:element name="price" type="xs:decimal"/>
      </xs:sequence>
      <xs:attribute name="id" type="xs:string"/>
    </xs:complexType>
  </xs:element>
</xs:schema>
```

What XSD gives you

- **Data types** — string, decimal, date, and custom types.
- **Structure** — required elements, order, and cardinality.
- **Constraints** — patterns, ranges, and enumerations.
- **Namespaces** — first-class support for qualified names.

Validating in Java

The `javax.xml.validation` API loads a schema and validates a document, throwing on the first violation — you enforce the contract before processing untrusted input.

© 2026 ByteMart Digital

```java
Schema schema = SchemaFactory
    .newInstance(XMLConstants.W3C_XML_SCHEMA_NS_URI)
    .newSchema(new File("book.xsd"));
schema.newValidator().validate(new StreamSource(xmlFile));
```

Describe and enforce structure

XSD defines the allowed elements, types, order, and constraints of valid XML, with rich data types and namespace support. A validating parser enforces the contract before your app processes the document.

MODULE 4 · XML FUNDAMENTALS

CHAPTER 17

Namespaces in Depth

Avoiding name collisions when combining XML vocabularies.

When one document mixes elements from different vocabularies — say, your data plus a standard like SOAP — name collisions become inevitable. **Namespaces** qualify names so they never clash.

The problem they solve

Two vocabularies might both define a `<title>` element meaning different things. A namespace binds elements to a unique URI, so a parser can tell them apart even when the local names match.

Declaring and using

xml

```
<catalog xmlns="http://bytemart.com/books"
         xmlns:meta="http://bytemart.com/meta">
  <book>
    <title>XML Basics</title>
    <meta:tag>education</meta:tag>
  </book>
</catalog>
```

`xmlns` declares a **default namespace** for unprefixed elements; `xmlns:meta` binds the prefix `meta` to another namespace. The URI is an identifier, not a location — nothing is fetched from it.

Prefixes are local

WATCH OUT

A namespace **prefix** is just a local alias for the URI — two documents can use different prefixes for the same namespace. Always compare by the namespace **URI**, never by the prefix string.

Namespaces in Java

Parsers, XPath, and JAXB are all namespace-aware. Enable namespace awareness on your parser (it is off by default in some APIs), and bind prefixes to URIs when writing XPath against namespaced documents.

Qualify names by URI

Namespaces bind element names to unique URIs so vocabularies can coexist without collisions. Prefixes are local aliases — always compare by the namespace URI, and enable namespace awareness in your parser.

© 2026 ByteMart Digital

MODULE 5 · XML IN JAVA

CHAPTER 18

Parsing XML: DOM, SAX & StAX

Three parsing models with very different memory and control trade-offs.

Java offers three ways to parse XML, and choosing the right one matters enormously for large documents. They differ in how much they load into memory and how much control you have.

The three models



DOM loads the whole tree; SAX pushes events; StAX lets you pull tokens.

Model	Style	Memory
DOM	whole tree in memory	high
SAX	push events to a handler	low
StAX	pull tokens as you read	low

DOM: the whole tree

DOM loads the entire document into an in-memory tree you can navigate and modify freely. It is the easiest to use — but for a large file it can exhaust memory.

```java
Document doc = DocumentBuilderFactory.newInstance()
    .newDocumentBuilder().parse(xmlFile);
NodeList titles = doc.getElementsByTagName("title");
```

SAX and StAX: streaming

SAX **pushes** events (start element, text, end element) to a handler you write — fast and memory-light, but read-only and stateful to code. StAX **pulls** tokens on demand, giving you the same streaming efficiency with more natural control flow.

Choosing

- **DOM** — small documents you need to navigate or edit.
- **SAX** — large documents, event-driven processing.
- **StAX** — large documents, when you want cursor-style control.

Tree for small, stream for big

DOM loads the whole document for easy navigation but uses memory; SAX (push) and StAX (pull) stream large documents with little memory. Pick by document size and whether you must edit.

© 2026 ByteMart Digital

MODULE 5 · XML IN JAVA

CHAPTER 19

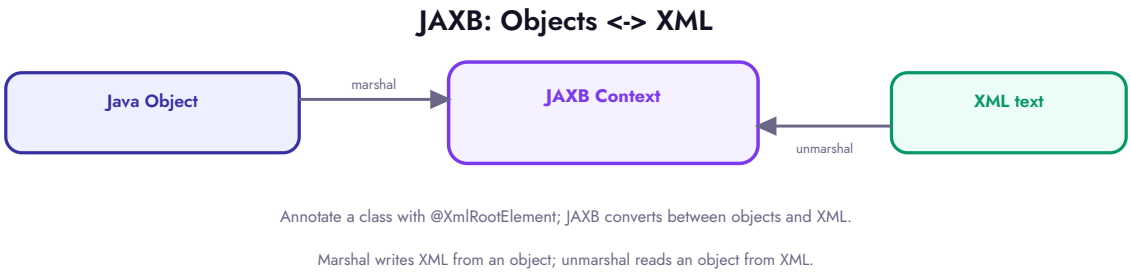
JAXB: XML Binding

Mapping XML to Java objects and back with annotations — the Jackson equivalent for XML.

19

Just as Jackson binds JSON to objects, **JAXB** (Jakarta XML Binding) binds XML to Java classes with annotations — turning verbose parsing code into a couple of method calls.

Marshalling and unmarshalling



JAXB marshals an annotated object to XML and unmarshals XML back into an object.

Annotating a class

```java
@XmlRootElement(name = "book")
@XmlAccessorType(XmlAccessType.FIELD)
public class Book {
    @XmlAttribute private String id;
    @XmlElement    private String title;
    @XmlElement    private BigDecimal price;
}
```

Converting

```java
JAXBContext ctx = JAXBContext.newInstance(Book.class);

// object -> XML
ctx.createMarshaller().marshal(book, System.out);

// XML -> object
Book b = (Book) ctx.createUnmarshaller()
    .unmarshal(new File("book.xml"));
```

The key annotations

- `@XmlRootElement` — marks the top-level element.

- `@XmlElement` / `@XmlAttribute` — map fields to elements or attributes.
- `@XmlAccessorType` — control field vs getter access.

NOTE

Since Java 11, JAXB is no longer bundled with the JDK — add the `jakarta.xml.bind` API and a runtime dependency. This trips up many upgrades from Java 8.

Annotations bind XML to objects

JAXB marshals annotated Java objects to XML and unmarshals back, replacing manual parsing. Mark classes with @XmlRootElement and fields with @XmlElement/@XmlAttribute — and add the dependency on Java 11+.

MODULE 5 · XML IN JAVA

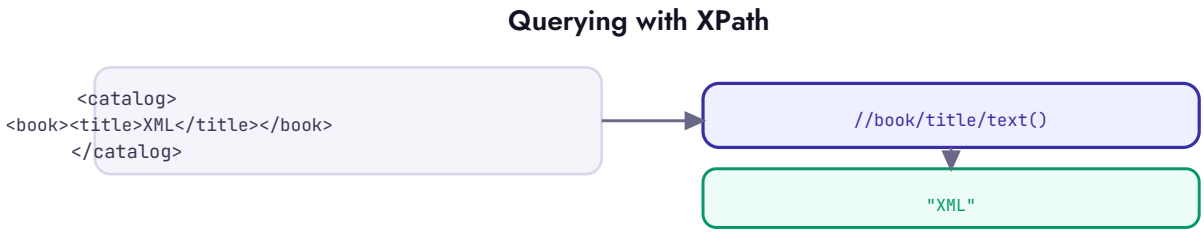
CHAPTER 20

Querying with XPath

Navigating and selecting nodes in an XML document with path expressions.

XPath is a powerful query language for XML — the model that inspired JSONPath. It navigates the document tree with path expressions to select exactly the nodes you want.

Selecting nodes



XPath navigates an XML tree with path expressions to select nodes and values.

XPath navigates an XML tree with path expressions to select nodes and values.

```text
/catalog/book/title      // absolute path
//title                  // all title elements, any depth
//book[@id='b1']         // book with attribute id = b1
//book[price>200]/title  // titles of books priced over 200
count(//book)            // a function: number of books
```

The syntax

Expression	Selects
/	the root / a step
//	descendants at any depth
@attr	an attribute
[predicate]	a filter condition
text()	the text content

XPath in Java

```java
XPath xpath = XPathFactory.newInstance().newXPath();
String title = xpath.evaluate(
    "/catalog/book[1]/title", document);
```

© 2026 ByteMart Digital

WATCH OUT

For namespaced documents, XPath needs a namespace context binding prefixes to URIs — a plain expression will not match qualified elements otherwise. This is a frequent source of 'why does my XPath return nothing' confusion.

Navigate the tree by path

XPath selects nodes and values from XML with path expressions, predicates, and functions — far more expressive than manual traversal. Remember to supply namespace bindings for namespaced documents.

© 2026 ByteMart Digital

MODULE 5 · XML IN JAVA

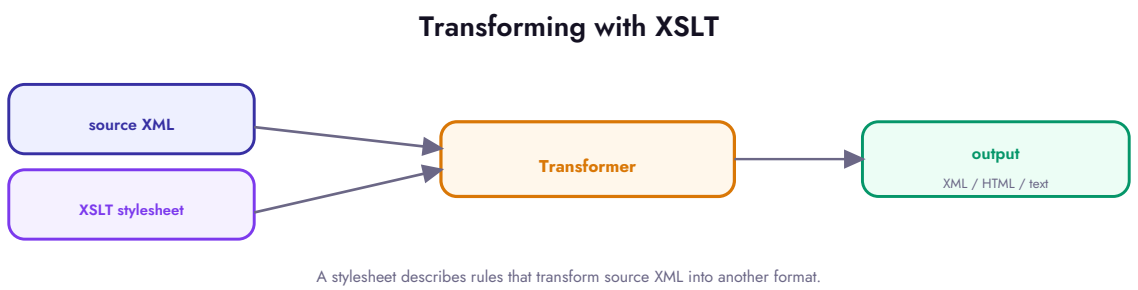
CHAPTER 21

Transforming with XSLT

Declaratively converting XML into other formats with stylesheets.

XSLT (eXtensible Stylesheet Language Transformations) transforms XML into another format — different XML, HTML, or plain text — using a declarative stylesheet of rules. It is a whole language dedicated to reshaping XML.

How it works



A stylesheet of rules transforms source XML into another format via a transformer.

An XSLT stylesheet is a set of **templates**, each matching part of the source and describing the output to produce for it. A transformer applies the stylesheet to the source document.

A small transformation

```xml
<xsl:stylesheet version="1.0"
    xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
  <xsl:template match="/catalog">
    <html><body>
      <xsl:for-each select="book">
        <p><xsl:value-of select="title"/></p>
      </xsl:for-each>
    </body></html>
  </xsl:template>
</xsl:stylesheet>
```

Applying it in Java

```java
Transformer t = TransformerFactory.newInstance()
    .newTransformer(new StreamSource("style.xslt"));
t.transform(new StreamSource("catalog.xml"),
            new StreamResult(System.out));
```

When to reach for XSLT

XSLT excels at document-to-document transformation — rendering XML as HTML, converting between XML vocabularies, or restructuring feeds. For simple extraction, XPath alone is enough; XSLT is for producing whole new documents.

Reshape XML declaratively

XSLT transforms XML into other XML, HTML, or text using template rules matched against the source. Reach for it when you need to produce a whole new document; use plain XPath for simple extraction.

© 2026 ByteMart Digital

MODULE 6 · INTEROP & PRACTICE

CHAPTER 22

JSON <-> XML Interop

Converting between the two formats, and the mismatches to watch for.

Real systems often speak both formats — a JSON API fronting an XML backend, or vice versa. Converting between them is common, but the two data models do not map perfectly.

Converting in Java

Jackson has an XML module (`jackson-dataformat-xml`) that reads and writes XML with the same `ObjectMapper` style, so one annotated class can be serialized to either format — the simplest bridge for object-based data.

```java
XmlMapper xmlMapper = new XmlMapper();
String xml = xmlMapper.writeValueAsString(book);

// same object, JSON
String json = new ObjectMapper().writeValueAsString(book);
```

The model mismatches

Concept	XML	JSON
Attributes	yes	no equivalent
Namespaces	yes	none
Order	significant	objects unordered
Repeated tags	natural	become arrays

WATCH OUT

Round-tripping is lossy. XML attributes, namespaces, comments, and mixed content have no clean JSON equivalent, so XML → JSON → XML rarely reproduces the original exactly. Convert with intent, not blindly.

A pragmatic approach

For structured, object-like data, map both formats to the same DTOs and let Jackson handle each. For document-centric XML with attributes and namespaces, a naive conversion loses meaning — handle it explicitly.

Bridge, but mind the gaps

Jackson's XML module lets one class serialize to JSON or XML. But the models differ — attributes, namespaces, and order have no JSON equivalent — so conversion is lossy. Map object-like data via DTOs and handle document XML deliberately.

© 2026 ByteMart Digital

MODULE 6 · INTEROP & PRACTICE

CHAPTER 23

Security: XXE & Injection

The parser vulnerabilities every developer handling untrusted XML and JSON must know.

Parsing untrusted data is a security-sensitive act. XML in particular has a notorious class of vulnerabilities that a default parser leaves wide open — you must harden it explicitly.

XML External Entity (XXE)

XML allows entities that reference external resources. A malicious document can define an entity pointing at a local file or internal URL, and a default parser will happily fetch it — leaking files or enabling server-side request forgery.

WATCH OUT

XXE is one of the most serious and common XML flaws. It can read arbitrary server files and reach internal systems. **Always** disable external entities and DTDs when parsing untrusted XML.

java

```java
DocumentBuilderFactory f = DocumentBuilderFactory.newInstance();
f.setFeature(
  "http://apache.org/xml/features/disallow-doctype-decl", true);
f.setExpandEntityReferences(false);
// now safe against XXE
```

Billion laughs

A related attack uses nested entity definitions that expand exponentially, exhausting memory (a denial of service). Disabling DTD processing defends against this too.

JSON risks

- **Deeply nested** or huge payloads can exhaust memory — enforce size and depth limits.
- **Jackson default typing** has enabled remote-code-execution gadget chains — never enable it.
- **Duplicate keys** and unexpected fields — validate against a schema.

The rule

Treat all incoming JSON and XML as hostile. Harden parsers, disable dangerous features, validate against a schema, and bound sizes — before the data reaches your business logic.

Harden parsers, distrust input

Untrusted XML must have external entities and DTDs disabled to prevent XXE and billion-laughs attacks; untrusted JSON needs size limits and no default typing. Validate everything at the boundary.

© 2026 ByteMart Digital

MODULE 6 · INTEROP & PRACTICE

CHAPTER 24

Performance & Best Practices

Efficient processing and a consolidated checklist for working with JSON and XML.

You now know both formats deeply. This closing chapter gathers the performance techniques and best practices that separate correct code from production-ready code.

Performance essentials

- **Reuse the mapper** — create one configured `ObjectMapper` (or `XmlMapper`) and share it; it is thread-safe and expensive to build.
- **Stream large data** — use Jackson streaming or SAX/StAX for big documents instead of loading everything.
- **Write to streams** — serialize straight to an `OutputStream`, not a giant in-memory string.
- **Bound sizes** — limit payload size and nesting depth on input.

Design best practices

- Expose **DTOs**, not entities, as your serialized contract.
- Validate untrusted input against a **schema** at the boundary.
- Be explicit about **nulls vs absent** fields in your API.
- Use a consistent **naming strategy** across the API.
- Version your formats so contracts can evolve safely.

Security recap

- Disable external entities and DTDs for untrusted XML (XXE).
- Never enable Jackson default typing.
- Never serialize secrets — `@JsonIgnore` them.

Choosing the format, one last time

Default to **JSON** for APIs and web/mobile clients — compact, fast, and universally supported. Reach for **XML** when you need documents, namespaces, rich schemas, transformation, or interoperability with enterprise standards.

> **Fast, safe, and clear**
> Reuse mappers, stream large data, and bound input sizes; expose DTOs, validate against schemas, and harden parsers. Default to JSON for APIs and XML for documents and enterprise standards.

© 2026 ByteMart Digital